ChatGPT dijo:

Se utiliza para proteger los formularios contra ataques de tipo **CSRF (Cross-Site Request Forgery)**.

En tu caso, como tenés un formulario de login, sirve para asegurarte de que la petición que llega a `login.php` proviene realmente de tu página y no de otro sitio.

¿Qué hace exactamente?

Esta línea:

PHP

```php
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
```

hace tres cosas:

1. `random_bytes(32)` genera **32 bytes aleatorios criptográficamente seguros**.
2. `bin2hex()` los convierte a una cadena hexadecimal.
3. Guarda esa cadena en la sesión del usuario.

Por ejemplo, podría generar algo parecido a:

Texto sin formato

```
8f3a6c1e7b2d9a4c5e8f1a3b6d7c9e0f...
```

Cada sesión tendrá su propio token.

Después lo enviás en el formulario

Por ejemplo:

PHP

```php
<input
    type="hidden"
    name="csrf_token"
    value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>"
>
```

Cuando el usuario presiona **Ingresar**, se envían:

Texto sin formato

```
email
passw
```

```
csrf_token
```

Y en `login.php` verificás:

```php
$token = $_POST['csrf_token'] ?? '';

if (
    empty($_SESSION['csrf_token']) ||
    !hash_equals($_SESSION['csrf_token'], $token)
) {
    header('Location: index.php?error=1');
    exit;
}
```

Es decir:

> **"El token que recibí del formulario tiene que ser exactamente el mismo que yo había guardado en la sesión."**

¿Qué problema evita?

Imaginá que alguien crea otra página maliciosa que intenta enviar automáticamente una petición a:

```
https://tudominio.com/login.php
```

o, más habitualmente, a una operación privada como:

```
https://tudominio.com/cambiar-email.php
```

El atacante no conoce el `csrf_token` almacenado en la sesión de tu usuario, por lo que la petición no pasa la validación.

Una aclaración importante para tu login

En **un formulario de login**, CSRF no es tan crítico como en operaciones posteriores como:

- cambiar contraseña;
- modificar datos personales;
- solicitar una constancia;
- realizar un trámite;

- modificar información de matrícula;
- realizar una operación administrativa.

Pero **no está mal incorporarlo desde el principio**, especialmente porque estamos diseñando ahora el portal de colegiados.

Para tu proyecto yo mantendría:

Texto sin formato

```
index.php
     ↓
login.php
     ↓
auth.php
     ↓
informacion.php
     ↓
otras páginas privadas
```

y usaría el CSRF principalmente en **todas las operaciones que modifican datos**, no solamente en el login.

Además, hay algo que te recomiendo hacer antes de avanzar: **separar** `auth.php`, `csrf.php` **y** `conexion.php`. Así evitamos terminar copiando código de seguridad en cada página del portal.